

Descomposición de Benders

Técnicas de Descomposición en Optimización

Problema Maestro · Subproblema · Cortes de Optimalidad y Factibilidad · Algoritmo Clásico · Callbacks de Gurobi · Extensiones

Por P. Hurtado · 2025

Índice

1. Motivación	1
Variables complicantes en MIP	2
2. Problema Maestro y Subproblema	2
Formulación general	2
Función de recourse	3
3. Cortes de Optimalidad y Factibilidad	3
Cortes de optimalidad	3
Cortes de factibilidad	4
Representación del problema completo	4
4. Algoritmo Clásico de Benders	4
Descripción del algoritmo	5
5. Aspectos Computacionales	5
Arranque en frío y calentamiento	5
Degeneración dual y cortes de Magnanti-Wong	6
6. Implementación con Callbacks de Gurobi	6
Lazy constraints vs. cortes en callback	6
Extracción de información dual de Gurobi	6
7. Análisis de Convergencia	7
Diagrama de convergencia	7
8. Extensiones	8
Generalized Benders Decomposition (Geoffrion, 1972)	8
Método L-shaped (Van Slyke & Wets, 1969)	8
Multi-cut Benders	8
9. Aplicaciones	9
Problema de Transporte con Costos Fijos (FCTP)	9
Planning de capacidad bajo incertidumbre	9
Diseño de redes de distribución	9

Sección 1: Motivación

Variables complicantes en MIP

Definición 1.1 · Variables complicantes

En un problema de optimización mixto-entero, las *variables complicantes* son aquellas que, al ser fijadas (en general, las variables binarias/enteras), dejan un subproblema más fácil de resolver (típicamente un LP). La idea de Benders es iterar sobre los valores de estas variables, resolviendo el subproblema resultante en cada iteración.

Ejemplo 1.1 · Localización-Transporte

En un problema de localización de instalaciones con costos de transporte, las variables $y_j \in \{0, 1\}$ (abrir instalación j) son las variables complicantes. Al fijar y^* , el subproblema de transporte en x es un LP de flujo, resoluble en tiempo polinomial.

La descomposición de Benders (Benders, 1962) es especialmente útil cuando: (1) las variables complicantes son pocas respecto al total de variables, y (2) el subproblema resultante tiene estructura explotable (LP, flujo en red, etc.).

Sección 2: Problema Maestro y Subproblema

Formulación general

Definición 2.1 · Problema original

Consideramos el problema:

$$\begin{aligned} \min_{x,y} \quad & c^\top x + f^\top y \\ \text{s.a.} \quad & Ax + By \geq b \\ & x \geq 0, \quad y \in Y \end{aligned} \tag{2.1}$$

donde $x \in \mathbb{R}^n$ son las variables continuas (de *recourse*) e $y \in Y \subseteq \mathbb{R}^p$ son las variables complicantes (típicamente enteras: $Y \subseteq \mathbb{Z}^p$).

Definición 2.2 · Subproblema $\text{SP}(y^*)$

Para un valor fijo $y^* \in Y$, el *subproblema* en x es:

$$\begin{aligned} \text{SP}(y^*) : \quad & \min_{x \geq 0} \quad c^\top x \\ \text{s.a.} \quad & Ax \geq b - By^* \end{aligned} \tag{2.2}$$

Este es un LP en x (asumiendo A, c fijas).

Definición 2.3 · Dual del subproblema $\text{DSP}(y^*)$

El dual de $\text{SP}(y^*)$ es:

$$\begin{aligned} \text{DSP}(y^*) : \quad & \max_{u \geq 0} \quad u^\top (b - By^*) \\ \text{s.a.} \quad & A^\top u \leq c \end{aligned} \tag{2.3}$$

La región factible del dual $\mathcal{U} = \{u \geq 0 : A^\top u \leq c\}$ es *independiente* de y^* .

Proposición 2.1 · Independencia del poliedro dual

El poliedro $\mathcal{U} = \{u \geq 0 : A^\top u \leq c\}$ no depende de y^* . Por lo tanto, sus puntos extremos $\{u^1, \dots, u^P\}$ y sus rayos extremos $\{r^1, \dots, r^Q\}$ son fijos para todas las iteraciones del algoritmo.

Demostración

La factibilidad del dual $A^\top u \leq c, u \geq 0$ depende únicamente de las matrices A y c , que no varían al cambiar y^* . Por lo tanto, la geometría del poliedro factible del dual es invariante. $\square$

Función de recourse**Definición 2.4 · Función de recourse $Q(y)$**

La *función de recourse* es:

$$Q(y) = \min_{x \geq 0} \{c^\top x : Ax \geq b - By\} = \max_{u \in \mathcal{U}} u^\top (b - By).$$

Proposición 2.2 · Convexidad de $Q(y)$

$Q(y)$ es una función *convexa* de y (sobre el dominio donde $\text{SP}(y)$ es factible).

Demostración

$Q(y) = \max_{u \in \mathcal{U}} u^\top (b - By)$ es el máximo de funciones afines en y , que es una función convexa. $\square$

La convexidad de $Q(y)$ implica que los cortes de Benders son planos de soporte de Q : cada corte es una cota inferior lineal válida de $Q(y)$ para todo y .

Sección 3: Cortes de Optimalidad y Factibilidad**Cortes de optimalidad****Definición 3.1 · Corte de optimalidad**

Para cada punto extremo óptimo $u^p \in \mathcal{U}$ de $\text{DSP}(y^k)$, el *corte de optimalidad* es:

$$\eta \geq (u^p)^\top (b - By) \quad \forall y \in Y,$$

donde η es la variable surrogate que representa el costo del subproblema en el problema maestro.

Proposición 3.1 · Validez del corte de optimalidad

El corte $(u^p)^\top (b - By) \leq \eta$ es válido: para todo y tal que $\text{SP}(y)$ es factible y acotado, $Q(y) \geq (u^p)^\top (b - By)$.

Demostración

Por dualidad fuerte LP: si $\text{SP}(y)$ es factible y acotado, $Q(y) = \max_{u \in \mathcal{U}} u^\top (b - By) \geq (u^p)^\top (b - By)$ para cualquier $u^p \in \mathcal{U}$. $\square$

En cada iteración solo se agrega el corte del punto extremo óptimo de $\text{DSP}(y^k)$. La selección del corte más violado (el que más aleja η de su valor actual) es la estrategia estándar.

Cortes de factibilidad

Definición 3.2 · Corte de factibilidad

Cuando $\text{DSP}(y^k)$ es no acotado (equivalentemente, $\text{SP}(y^k)$ es infactible), existe un rayo extremo $r^q \in \mathcal{U}$ tal que $(r^q)^\top(b - By^k) > 0$. El *corte de factibilidad* es:

$$(r^q)^\top(b - By) \leq 0 \quad \forall y \in Y.$$

Este corte elimina valores de y para los cuales $\text{SP}(y)$ es infactible.

Proposición 3.2 · Validez del corte de factibilidad

Si r^q es un rayo extremo de $\mathcal{U}$ (es decir, $A^\top r^q \leq 0$, $r^q \geq 0$) y $(r^q)^\top(b - By^k) > 0$, entonces para cualquier y tal que $\text{SP}(y)$ es factible se tiene $(r^q)^\top(b - By) \leq 0$.

Demostración

Por el lema de Farkas: $\text{SP}(y)$ es factible si y solo si para todo rayo extremo r del dual $(r)^\top(b - By) \leq 0$. Por contrareciproco, si $(r^q)^\top(b - By) > 0$ entonces $\text{SP}(y)$ es infactible. $\square$

△ Atención

En la práctica, cuando $\text{SP}(y^k)$ es infactible en Gurobi, se extrae el *dual de Farkas* (`constr.FarkasDual`) habilitando `model.setParam("InfUnbdInfo", 1)` antes de resolver. El vector resultante satisface $A^\top r \leq 0$, $r \geq 0$, y corresponde a un rayo extremo de $\mathcal{U}$.

Representación del problema completo

Teorema 3.3 · Descomposición de Benders (Benders, 1962)

El problema original (2.1) es equivalente al siguiente *problema maestro*:

$$\begin{aligned} \min_{y \in Y, \eta} \quad & f^\top y + \eta \\ \text{s.a.} \quad & \eta \geq (u^p)^\top(b - By) & \forall p \in \mathcal{P} \quad (\text{cortes optimalidad}) \\ & 0 \geq (r^q)^\top(b - By) & \forall q \in \mathcal{Q} \quad (\text{cortes factibilidad}) \\ & \eta \geq -\infty \end{aligned} \tag{3.1}$$

donde $\mathcal{P}$ es el conjunto de puntos extremos de $\mathcal{U}$ y $\mathcal{Q}$ el de rayos extremos.

Sección 4: Algoritmo Clásico de Benders

Descripción del algoritmo

Algoritmo de Benders (versión clásica)

```

1: Inicializar:  $LB \leftarrow -\infty$ ,  $UB \leftarrow +\infty$ ,  $\mathcal{P} \leftarrow \emptyset$ ,  $\mathcal{Q} \leftarrow \emptyset$ ,  $k \leftarrow 1$ 
2: repeat
3:   Resolver RMP (Restricted Master Problem): obtener  $y^k$ ,  $\eta^k$ 
4:    $LB \leftarrow f^\top y^k + \eta^k$ 
5:   Resolver SP( $y^k$ ) y su dual  $DSP(y^k)$ 
6:   if  $DSP(y^k)$  es acotado (SP factible) then
7:      $u^k \leftarrow$  solución óptima de  $DSP(y^k)$ 
8:      $UB \leftarrow \min(UB, f^\top y^k + (u^k)^\top (b - By^k))$ 
9:     Agregar corte de optimalidad:  $\eta \geq (u^k)^\top (b - By)$ 
10:     $\mathcal{P} \leftarrow \mathcal{P} \cup \{u^k\}$ 
11:   else
12:      $r^k \leftarrow$  rayo extremo de  $DSP(y^k)$  (dual de Farkas)
13:     Agregar corte de factibilidad:  $(r^k)^\top (b - By) \leq 0$ 
14:      $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{r^k\}$ 
15:   end if
16:    $k \leftarrow k + 1$ 
17: until  $|UB - LB| \leq \varepsilon \cdot |LB|$ 
18: return  $y^k$ ,  $x^* =$  solución óptima de  $SP(y^k)$ 

```

Proposición 4.1 · Monotonidad de las cotas

La cota inferior LB es *no decreciente* a lo largo de las iteraciones (se agregan restricciones al RMP, que solo puede empeorar o mantener el objetivo del maestro). La cota superior UB puede no ser monotónica si y^k cambia entre iteraciones.

Teorema 4.2 · Convergencia finita

El algoritmo de Benders converge en un número finito de iteraciones, dado que el número de puntos extremos y rayos extremos de $\mathcal{U}$ es finito (aunque potencialmente exponencial).

Sección 5: Aspectos Computacionales

Arranque en frío y calentamiento

El RMP inicial (sin cortes) es muy débil: η está sin restricción inferior, lo que lleva a convergencia lenta. Estrategias de warm-start: (1) resolver la relajación LP del problema original y usar las soluciones duales como cortes iniciales; (2) aplicar heurísticas para generar varios y^0 buenos y agregar sus cortes antes del primer ciclo.

Degeneración dual y cortes de Magnanti-Wong

△ Atención

Cuando $DSP(y^k)$ tiene múltiples soluciones óptimas (degeneración dual), los distintos puntos extremos óptimos generan cortes de diferente fortaleza para el RMP actual. Un corte débil lleva a convergencia lenta.

Definición 5.1 · Corte de Pareto-óptimo (Magnanti & Wong, 1981)

Entre todos los puntos extremos óptimos de $DSP(y^k)$, se elige aquel que maximiza $(u)^\top (b - By^0)$ donde y^0 es un punto interior del convéx hull de soluciones factibles del maestro (e.g., el promedio de soluciones anteriores). Este corte es el más fuerte respecto al interior del maestro.

En la práctica, calcular el corte de Magnanti-Wong requiere resolver un LP auxiliar en u con la restricción de que u sea óptima dual de $DSP(y^k)$ y maximice $(u)^\top (b - By^0)$. El esfuerzo extra suele compensarse con una convergencia significativamente más rápida.

Sección 6: Implementación con Callbacks de Gurobi

Lazy constraints vs. cortes en callback

Definición 6.1 · Callback de Gurobi

Gurobi permite intervenir durante el proceso de Branch and Bound mediante la función `model.optimize(callback)`. El argumento es una función (o clase callable) que recibe información del nodo actual y puede agregar cortes.

El punto de entrada más importante para cortes de Benders es `GRB.Callback.MIPSOL`: se activa cada vez que Gurobi encuentra una solución entera candidata. En ese punto se resuelve $SP(y^k)$ y se agregan los cortes correspondientes como lazy constraints.

Proposición 6.1 · Protocolo de callbacks para Benders

Para implementar Benders con callbacks en Gurobi:

1. Configurar `master.setParam("LazyConstraints", 1)` antes de `optimize()`.
2. Dentro del callback, cuando `self.where == GRB.Callback.MIPSOL`: extraer y^k con `self.cbGetSolution(y_vars)`, resolver $SP(y^k)$, y agregar cortes con `self.cbCut(lhs_expr, GRB.GREATER_EQUAL, rhs)`.

Extracción de información dual de Gurobi

Definición 6.2 · Variables duales y rayo de Farkas

Para un LP resuelto en Gurobi:

- `constr.Pi`: precio sombra = variable dual u_i .
- `constr.FarkasDual`: componente del rayo de Farkas r_i (solo disponible si `model.setParam("InfUnbdInfo", 1)` fue configurado antes de `optimize()` y el modelo es infactible).
- `var.RC`: costo reducido.

△ Atención

`InfUnbdInfo = 1` debe configurarse *antes* de llamar a `optimize()`. Si el modelo es infactible y `InfUnbdInfo = 0`, Gurobi no calcula el certificado de infactibilidad y `FarkasDual` retorna `None`.

Ejemplo 6.1 · Esqueleto de callback para Benders en Python

Listing 1: Callback de Benders con Gurobi

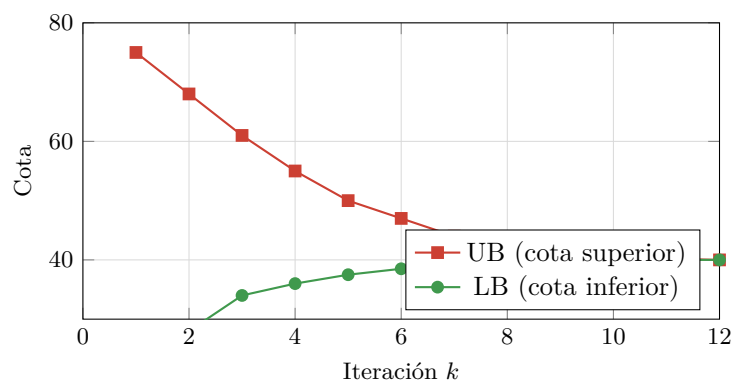
```

1 import gurobipy as gp
2 from gurobipy import GRB
3
4 class BendersCallback:
5     def __init__(self, y_vars, eta_var, data):
6         self.y_vars = y_vars
7         self.eta_var = eta_var
8         self.data = data
9         self.n_opt_cuts = 0
10        self.n_feas_cuts = 0
11
12    def __call__(self):
13        if self.where == GRB.Callback.MIPSOL:
14            y_val = {i: self.cbGetSolution(self.y_vars[i])
15                    for i in self.data['I']}
16            eta_val = self.cbGetSolution(self.eta_var)
17            obj_sp, u, r = self._solve_subproblem(y_val)
18            if obj_sp == float('inf'): # infactible
19                expr = gp.LinExpr()
20                # construir  $(r^q)^T (b - By) \leq 0$ 
21                self.cbCut(expr, GRB.LESS_EQUAL, 0)
22                self.n_feas_cuts += 1
23            elif obj_sp > eta_val + 1e-4: # corte violado
24                expr = gp.LinExpr()
25                # construir  $\eta \geq (u^p)^T (b - By)$ 
26                self.cbCut(expr, GRB.GREATER_EQUAL, rhs)
27                self.n_opt_cuts += 1
28
29    def _solve_subproblem(self, y_val):
30        sp = gp.Model()
31        sp.setParam("OutputFlag", 0)
32        sp.setParam("InfUnbdInfo", 1)
33        # ... construir SP(y_val) ...
34        sp.optimize()
35        if sp.Status == GRB.INFEASIBLE:
36            r = [c.FarkasDual for c in sp.getConstrs()]
37            return float('inf'), None, r
38        u = [c.Pi for c in sp.getConstrs()]
39        return sp.ObjVal, u, None
40
41 # Uso:
42 # master.setParam("LazyConstraints", 1)
43 # cb = BendersCallback(y_vars, eta_var, data)
44 # master.optimize(cb)

```

Sección 7: Análisis de Convergencia

Diagrama de convergencia



Proposición 7.1 · Monotonidad de LB

La cota inferior LB es no decreciente porque en cada iteración se agregan restricciones al RMP, lo que solo puede aumentar o mantener el objetivo del RMP.

El criterio de parada $|UB - LB| \leq \varepsilon \cdot |LB|$ garantiza una $(1 + \varepsilon)$ -optimalidad. En la práctica se usan $\varepsilon = 10^{-4}$ a 10^{-6} .

Sección 8: Extensiones**Generalized Benders Decomposition (Geoffrion, 1972)****Definición 8.1 · GBD**

Cuando el subproblema $SP(y)$ es un *programa convexo* (no necesariamente LP), la descomposición generalizada de Benders reemplaza los cortes lineales por *cortes de soporte convexo*:

$$\eta \geq Q(y^k) + \nabla Q(y^k)^\top (y - y^k),$$

donde $\nabla Q(y^k)$ es el subgradiente de Q en y^k (obtenido del multiplicador dual del subproblema convexo).

△ Atención

GBD requiere que el subproblema sea convexo (y diferenciable para usar ∇Q). No aplica directamente si $SP(y)$ es un MIP.

Método L-shaped (Van Slyke & Wets, 1969)**Definición 8.2 · Método L-shaped**

Extensión de Benders para *programas estocásticos de dos etapas*: la primera etapa decide y (e.g., inversión en capacidad) y la segunda etapa decide x^s para cada escenario s con probabilidad p_s . El problema de segunda etapa es:

$$Q(y) = \sum_s p_s Q_s(y), \quad Q_s(y) = \min_{x_s \geq 0} \{c_s^\top x_s : A_s x_s \geq b_s - B_s y\}.$$

Proposición 8.1 · Corte de optimalidad del L-shaped

Para una solución y^k , el corte de optimalidad agrega:

$$\eta \geq \sum_s p_s [(u^{k,s})^\top (b_s - B_s y^k) - (u^{k,s})^\top B_s (y - y^k)] = \sum_s p_s (u^{k,s})^\top (b_s - B_s y),$$

donde $u^{k,s}$ es la solución dual de $SP_s(y^k)$. Los escenarios son *independientes*: se resuelven en paralelo.

Multi-cut Benders**Definición 8.3 · Multi-cut**

En lugar de una única variable surrogate η , el *multi-cut* usa una variable η_s por escenario y agrega cortes individuales:

$$\eta_s \geq (u^{k,s})^\top (b_s - B_s y) \quad \forall s, \forall k.$$

El objetivo del maestro es $f^\top y + \sum_s p_s \eta_s$.

Multi-cut genera cortes más finos por iteración pero aumenta el tamaño del RMP. El trade-off depende del número de escenarios: para $|S|$ grande, el RMP crece rápidamente y puede volverse el cuello de botella.

Sección 9: Aplicaciones

Problema de Transporte con Costos Fijos (FCTP)

Ejemplo 9.1 · FCTP — Aplicación completa

Considere $I = 3$ orígenes, $J = 4$ destinos, con costos fijos f_{ij} de abrir la ruta (i, j) y costos variables de transporte c_{ij} . Las variables binarias $y_{ij} \in \{0, 1\}$ son las complicantes. Al fijar y^* , el subproblema es un LP de transporte clásico. A continuación se muestra una tabla de iteraciones típica:

Iter.	LB	UB	Gap (%)	Tipo de corte
1	$-\infty$	1248	—	Optimalidad
2	1102	1210	9.1	Optimalidad
3	1150	1195	3.9	Optimalidad
4	1170	1182	1.0	Optimalidad
5	1178	1178	0.0	Convergido

Planning de capacidad bajo incertidumbre

Ejemplo 9.2 · Two-stage stochastic planning

Primera etapa: decidir qué instalaciones construir ($z_i \in \{0, 1\}$, costo fijo f_i , capacidad cap_i). Segunda etapa: para cada escenario de demanda d^s , asignar demanda a instalaciones ($x_{ij}^s \geq 0$, costo unit. c_{ij}). Si la demanda no puede ser satisfecha en el escenario s , el subproblema es infactible y se agrega un corte de factibilidad que obliga a abrir más instalaciones o aumentar capacidad.

Diseño de redes de distribución

En problemas de diseño de redes, las variables de diseño de arcos son las complicantes. Al fijarlas, el subproblema de flujo es un LP separable por commodity (o por escenario en el caso estocástico). Los cortes de Benders se construyen directamente de los precios sombra del subproblema de flujo.

Referencias principales:

- Benders, J.F. (1962) — “Partitioning procedures for solving mixed-variables programming problems”. *Numerische Mathematik*.
- Geoffrion, A.M. (1972) — “Generalized Benders Decomposition”. *Journal of Optimization Theory and Applications*.
- Van Slyke, R. & Wets, R. (1969) — “L-shaped linear programs”. *SIAM Journal on Applied Mathematics*.
- Magnanti, T.L. & Wong, R.T. (1981) — “Accelerating Benders decomposition: algorithmic enhancement and model selection criteria”. *Operations Research*.
- Rahmiani et al. (2017) — “The Benders Decomposition Algorithm: A Literature Review”. *European Journal of Operational Research*.